

EC4.202
Introduction to Robotics: Perception and Planning
chips: a robot waiter

Gauri Krishnan Kshitij Patkar Harry Jain
2024102073 2024112001 2024112005

Spring 2026

Contents

1 Proposal	2
2 Objectives	2
3 Overview	2
4 Pipeline	3
4.1 Prerequisite SLAM	3
4.2 Nav2 Dual Planning	3
4.3 Localisation	4
5 Implementation	4
5.1 Simulation Setup	4
5.2 Mapping the Environment	5
5.3 Basic Navigation	7
5.4 Manual Waypoints	8
5.5 Autonomous Waiter Logic	9
5.6 Route Optimisation	10
5.7 Dynamic Obstacles	11
6 Conclusions	12

1 Proposal

Waypoint Navigation in a Dynamic Environment

In this project, in a restaurant setting, our waiter robot *chips* (can help if people starve) navigates autonomously in real-time through both static and dynamic obstacles to deliver dishes to designated tables (waypoints).

2 Objectives

- **Waypoint Navigation**
Waypoint based autonomous navigation should be implemented to enable *chips* to travel between the kitchen (pick-up point) and the designated tables (drop-off points) within the Gazebo restaurant environment. For each target waypoint, *chips* must first orient itself to the specified pose before proceeding to the next delivery location.
- **Efficient Sequencing**
Given a set of selected tables and the kitchen as the starting point, *chips* should compute a sequence that minimises overall travel distance and time within the environment, and return to the kitchen after completing the mission.
- **Dynamic Obstacle Avoidance During Delivery**
While navigating between waypoints, *chips* must be able to detect and avoid dynamic obstacles, such as customers or staff moving within the restaurant environment.

3 Overview

This section provides a brief mention and use of the key tools and software used in this project:

- **ROS2 Humble:**
The core robot middleware, providing communication between nodes, managing topics, action servers, and managing robot behaviour.
- **TurtleBot3:**
The simulated robot platform used for mapping, navigating and mission execution. We use the Waffle model equipped with a 360° Lidar, and will refer to this bot as *chips*.
- **Gazebo:**
The 3D physics simulation environment where *chips*, static, and dynamic obstacles are modelled and tested.
- **Google Cartographer:**
The tool performing SLAM (Simultaneous Localisation and Mapping) to build a map of the environment as *chips* explores. We run SLAM initially to build the map of the environment which is then exported as `.yaml` and `.pgm` files.
- **Navigation2:**
The navigation stack enabling autonomous path planning, localisation, and obstacle avoidance using the generated map and live sensor feed.

- **RViz2:**
Visualisation tool for monitoring the state of *chips*, sensor data, and mapping progress.
- **chips_go.py:**
A Python script that allows the user to select which tables to serve, determines the most efficient order, and sends them as waypoints to the Nav2 navigation stack.
It manages the overall delivery by providing status updates to the user and monitoring the progress of *chips* throughout the mission.
The program also ensures that *chips* starts the mission from the kitchen (pick-up point) and returns to the kitchen after completing all deliveries.

4 Pipeline

Here we explain the architecture that enables *chips* to work, understanding the conversion of raw sensor data to autonomous decision making.

4.1 Prerequisite SLAM

Before any autonomous navigation can happen, *chips* must have a spatial memory of its environment. We use Google Cartographer to perform SLAM for this, because Nav2 requires a static occupancy grid (the `.yaml` and `.pgm` files) to act as a truth map. Without this, the global planner has no way to calculate a route around walls and permanent obstacles.

4.2 Nav2 Dual Planning

Once the static map is loaded, Nav2 controls *chips*'s movement using a stacked approach of costmaps and planners.

1. The Global Planner:

When a table coordinate is sent, the global planner checks the static map exported from SLAM. From this, it creates a global costmap, which identifies the most efficient theoretical path to the table. This path is static and assumes the restaurant is empty and only accounts for the walls and tables captured during the initial mapping.

2. The Local Planner:

While the global planner handles the overall route, the local planner (using the DWB controller) is responsible for the actual movement commands. It creates a local costmap, which is a small, high frequency region around *chips* that is updated constantly using live Lidar data. If a customer walks in front of *chips* or a chair falls, the local costmap detects these as new obstacles not present on the original map. The local planner tries to follow the global path while simultaneously calculating a velocity search to steer around this new obstacles.

4.3 Localisation

For these two systems to work, *chips* needs to know where it is on the static map. We use AMCL (Adaptive Monte Carlo Localisation) for this to find an estimated position on the map.

5 Implementation

Here we explain the steps we took to develop and test *chips*. We divided the project into phases and ensured every phase was fully functional before moving on to the next one.

5.1 Simulation Setup

We begin our simulation in a test world in Gazebo, as this is a low resource way of providing a proof-of-concept for the working of the project. It lets us focus completely on implementing the actual logic without being bottlenecked by 3D rendering lag.

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

This starts the physics engine (`gzserver`) and the visualisation (`grclient`), and loads the default `turtlebot3_world` with *chips* in it, which provides a series of obstacles necessary for testing navigation.

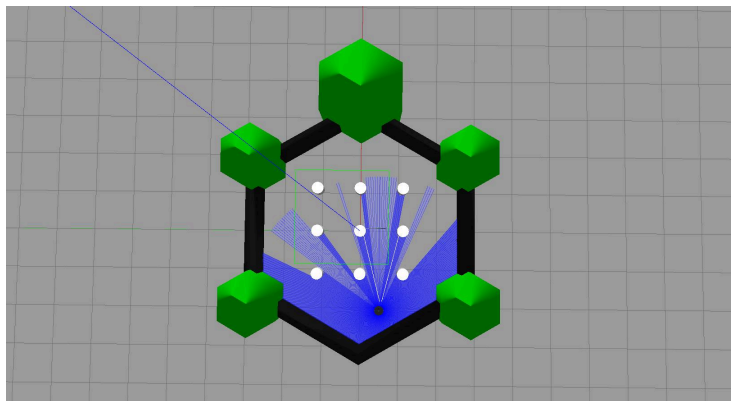


Figure 1: The default `turtlebot3_world` world with *chips* spawned in it.

We completed all the next steps explained in this default environment first to verify that everything works, and then created a custom Gazebo world `restaurant.world` to look like an actual restaurant with kitchen areas, tables, narrow corridors, etc. We will be demonstrating all the further steps in this custom environment.

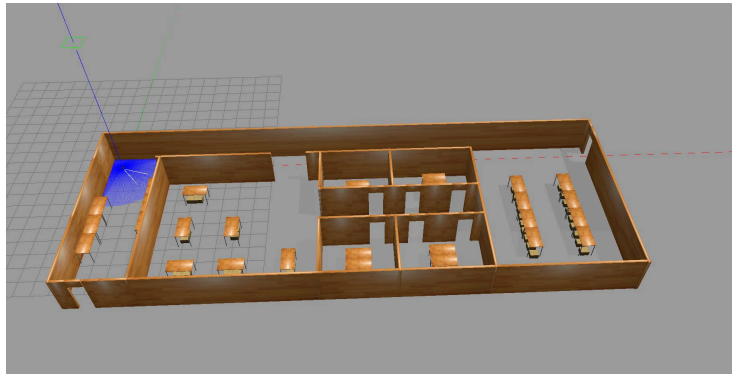


Figure 2: A perspective view of our custom restaurant world with *chips* spawned in it.

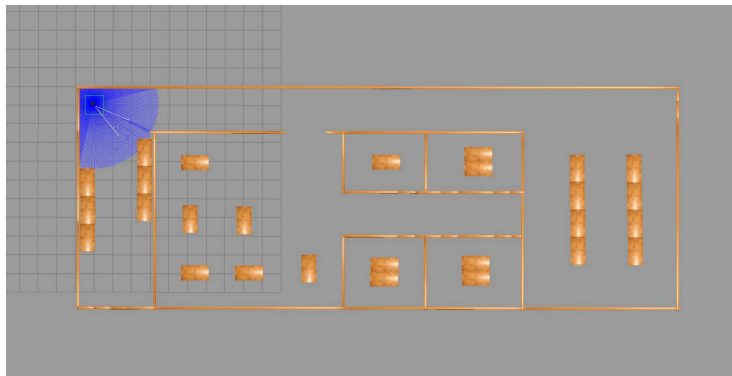


Figure 3: An orthographic view of our custom restaurant world with *chips* spawned in it.

As with the default environment, we created a custom launch file `rest_launch.py` to do the same as the earlier launch file, and make sure that *chips* spawns correctly with its Lidar working.

```
ros2 launch ~/src/rest_launch.py
```

5.2 Mapping the Environment

Before *chips* can navigate autonomously, it requires a static map of its environment. We achieve this by running SLAM by launching the `slam_toolbox` node.

```
ros2 launch turtlebot3_cartographer cartographer.launch.py \
  use_sim_time:=True
```

This begins processing Lidar data to build a 2D occupancy grid.

`use_sim_time:=True` tells the ROS nodes to synchronise with Gazebo's internal clock instead of the computer system clock, avoiding data lag.

Since *chips* can't map what it can't see, we drive it around manually using `teleop_keyboard`.

```
ros2 run turtlebot3_teleop teleop_keyboard
```

This listens for keyboard inputs and publishes messages to the `/cmd_vel` topic, controlling *chips*'s linear and angular velocity.

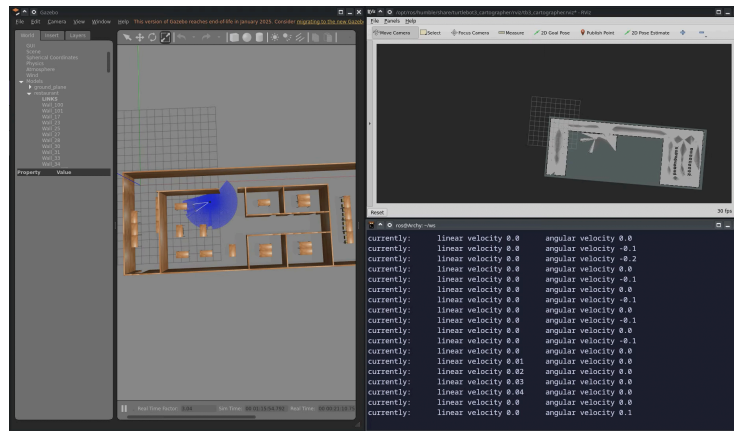


Figure 4: Performing SLAM by moving *chip* around using `teleop`. Left: the Gazebo environment. Top-right: the map being made in RViz2. Bottom-right: the `teleop` controls for moving *chips*.

As *chips* moves, the Lidar scans the (static) obstacles and physical boundaries, generating the 2D occupancy grid in RViz2. Once the map looks closed and complete, we freeze this data into a file that the navigation stack can read later.

```
ros2 run nav2_map_server map_saver_cli -f ~/ws/src/maps/rest_map
```

This calls the map server to take the current `/map` topic and save it as two files: `rest_map.pgm` (the image) and `rest_map.yaml` (the metadata) at the specified location (assuming that the directory exists).

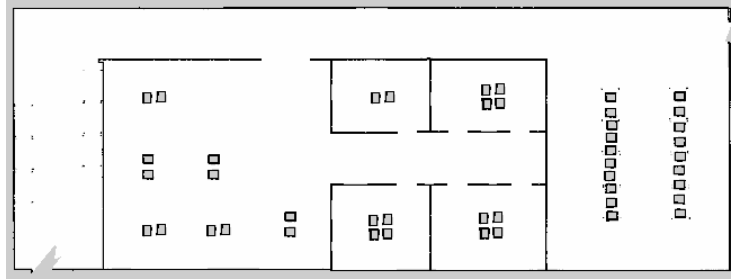


Figure 5: The final SLAM map. Black cells are obstacles and white cells are navigable area.

5.3 Basic Navigation

After obtaining the static map, we switch from manual to autonomous navigation. For this, we launch the navigation (nav2) stack.

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py \
  use_sim_time:=True \
  map:=/home/ros/ws/src/maps/rest_map.yaml
```

This loads the static map using the `map` argument which is used as the floor plan for the calculations, starts the AMCL node to track *chips*'s position and the planner/controller nodes to move the robot. Once RViz2 starts, we provide an initial pose required by AMCL using the 2D Pose Estimate in RViz2 based on *chips*'s current location in the Gazebo world.

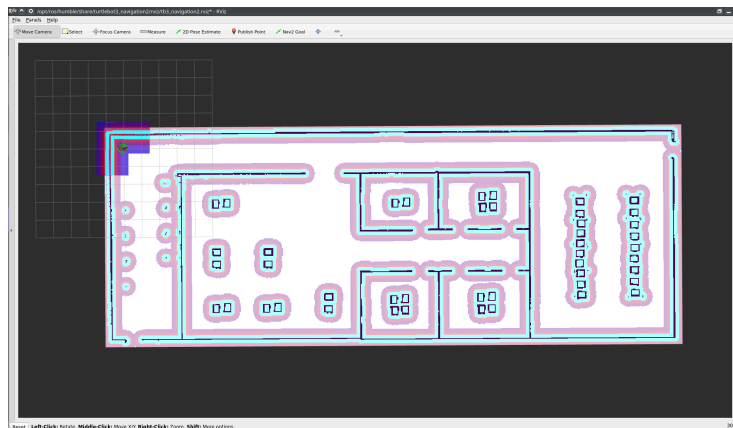


Figure 6: Nav2 initialised in RViz2 with the generated static map, after giving an initial pose estimate.

We test basic point-to-point navigation using the Nav2 Goal tool in RViz2 by manually setting goal poses by clicking and dragging on the map. This confirms that the global planner could successfully generate a path and the local planner could drive *chips* to a single destination.

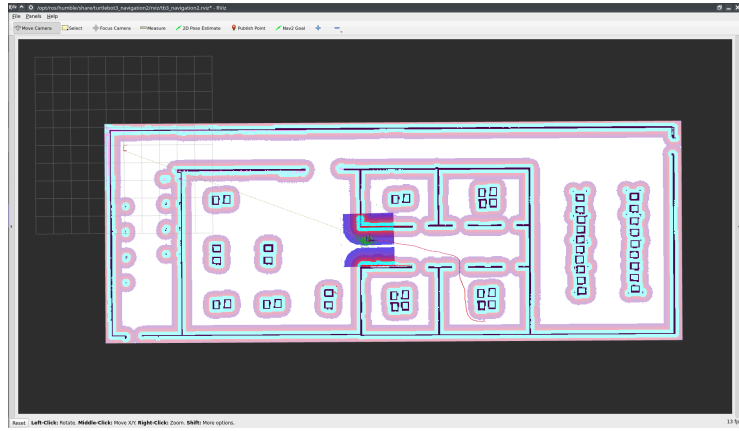


Figure 7: *chips* navigating to a goal pose set by clicking in RViz2.

5.4 Manual Waypoints

To simulate visiting multiple tables in a single go, we use the built-in RViz2 Waypoint tool by manually choosing several poses in a sequence by clicking on the map. We then feed these into Nav2's Waypoint Follower, which moves *chips* in the given sequence.

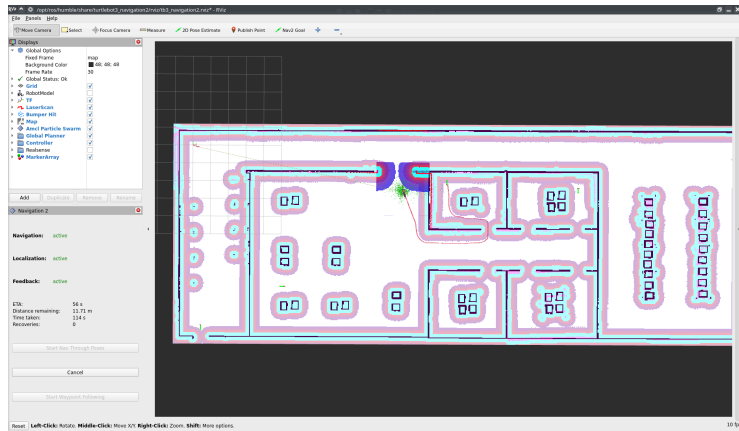


Figure 8: *chips* following waypoints (shown by green arrows) set by clicking in RViz2.

It is important to clarify our choice of Waypoint Follower over the other Navigate Through Poses mode in RViz2, as the latter treats the waypoints as a spline, curving through them continuously without stopping. Waypoint Follower forces *chips* to come to a complete stop at each specified point, like a waiter at a table should.

5.5 Autonomous Waiter Logic

To remove the need for manual clicking, we wrote a Python script (`chips_go.py`) to act as the brain, which uses the `nav2_simple_commander` API to control the robot.

The script has a hardcoded dictionary which maps human-readable labels (like `kitchen`, `table_1`, etc.) to their exact (x, y, θ) coordinates on our saved map.

```
TABLES = {
    "table_1": (5.464768886566162, -3.9170689582824707, 0.0),
    "table_2": (5.95006799697876, -6.284769058227539, 0.0),
    "table_3": (5.418345928192139, -8.252248764038086, 0.0),
    # ...
    "table_12": (28.020605087280273, -5.678124904632568, 0.0),
    "kitchen": (0.0, 0.0, 0.0),
}
```

We got these coordinates by listening to the `/clicked_point` topic, which gives information about the latest clicked point from RViz2's Publish Point option.

```
ros2 topic echo /clicked_point
```

The script provides a CLI (Command Line Interface) for the user to input a sequence of requested tables (like `1`, `5`, `2` or `all`). Once the input is taken, the script does the following:

1. Forces *chips* to navigate to the `kitchen` coordinate.
2. Converts the selected coordinates to a list of ROS 2 `PoseStamped` messages and sends them to the Nav2 action server via `navigator.followWaypoints()`.
3. On receiving a `TaskResult.SUCCEEDED` callback, it automatically commands *chips* to return to the `kitchen` to wait for the next order.

To help visualisation, the script also publishes to a `/waypoints` topic using `visualization_msgs/MarkerArray`. For every selected table, it renders a red 3D sphere at the coordinate with a text label floating above it showing its visit order (like `1: table_3`). This show us the intended path before it even moves.

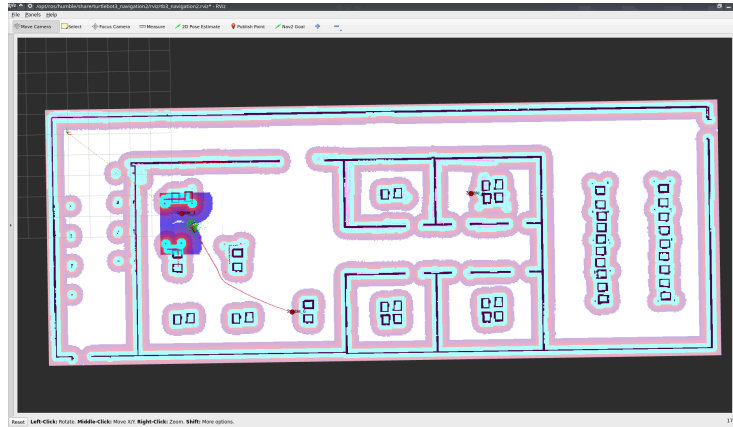


Figure 9: *chips* following the waypoints set by giving a sequence of table numbers. The waypoints are labelled and marked with red spheres.

5.6 Route Optimisation

For a given sequence of tables, navigating in that exact order is often highly inefficient, causing *chips* to zigzag around the restaurant. To fix this, we implement a custom route optimisation algorithm before sending the waypoints to Nav2.

We use a greedy nearest-neighbour approach. The algorithm initialises the `current` position as the kitchen, and calculates the 2D Euclidean distance $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ between the kitchen and all requested tables. It selects the closest table and sets it as the first waypoint, and updates `current` to that table, repeating the process for the remaining unvisited tables until the list is empty.

```
for name, pos in remaining.items():
    dist = euclidean_distance(current, pos[:2])
    if dist < nearest_dist:
        nearest_dist = dist
        nearest_name = name

ordered.append((nearest_name, remaining[nearest_name]))
current = remaining[nearest_name][:2]
del remaining[nearest_name]
```

To run the script, we simply run

```
python3 ~/src/chips_go.py
```

which launches the CLI to fetch the sequence of tables until the user agrees to there being no more orders.

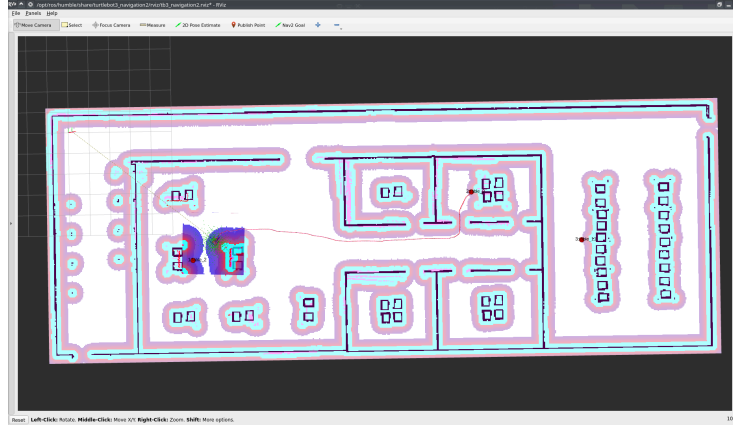


Figure 10: *chips* following an optimised sequence for the waypoints set by giving a sequence of table numbers. The waypoints are labelled and marked with red spheres.

5.7 Dynamic Obstacles

While our script (`chips_go.py`) was running, we introduced dynamic obstacles by adding objects into the Gazebo world in real-time. Because Nav2 maintains a local costmap that updates dynamically, *chips* was able to detect these new obstacles using its Lidar and steered itself around the obstacle without stopping the overall mission.

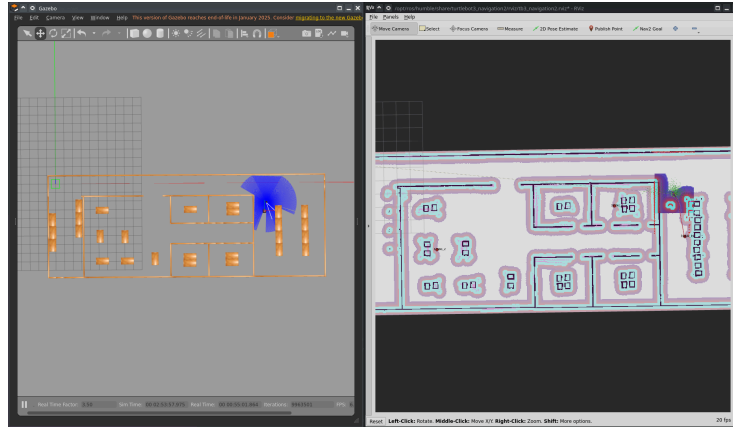


Figure 11: *chips* avoiding a construction cone placed in its path to the next waypoint.

It is important to note that the sequence of delivery waypoints is determined before the mission begins using an optimisation method based on the known map of the environment, which consists of the static layout of the restaurant, including walls, furniture, and other fixed obstacles.

Once this sequence is generated, the dishes are loaded onto *chips* in the same delivery order, allowing it to serve each table sequentially without rearranging items during the mission.

During execution, when *chips* encounters dynamic obstacles, they are handled through local path replanning rather than reordering the delivery sequence.

6 Conclusions

The project successfully demonstrated a fully integrated autonomous service robot, *chips*, capable of navigating a simulated restaurant environment with high precision and logical efficiency.

We achieved:

- *chips* reliably reaching all user-selected table waypoints.
- *chips* finding the optimised delivery route using a greedy nearest-neighbour algorithm.
- *chips* maintaining its mission even when the environment changes.
- *chips* making sure it always returns to and starts from the kitchen home base, like in a real deployment.